



King's Research Portal

Document Version
Peer reviewed version

[Link to publication record in King's Research Portal](#)

Citation for published version (APA):

Armony, M., Meroño-Peñuela, A., & Canal, G. (in press). TP-ONT: An Ontology for Enhanced Reasoning in AI Task Planning via Atomic Decomposition. In *The Semantic Web – ISWC 2026*

Citing this paper

Please note that where the full-text provided on King's Research Portal is the Author Accepted Manuscript or Post-Print version this may differ from the final Published version. If citing, it is advised that you check and use the publisher's definitive version for pagination, volume/issue, and date of publication details. And where the final published version is provided on the Research Portal, if citing you are again advised to check the publisher's website for any subsequent corrections.

General rights

Copyright and moral rights for the publications made accessible in the Research Portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognize and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the Research Portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the Research Portal

Take down policy

If you believe that this document breaches copyright please contact librarypure@kcl.ac.uk providing details, and we will remove access to the work immediately and investigate your claim.

TP-ONT: An Ontology for Enhanced Reasoning in AI Task Planning via Atomic Decomposition

Ma'ayan Armony^[0009-0007-5742-4452], Albert Meroño-Peñuela^[0000-0003-4646-5842], and Gerard Canal^[0000-0002-6718-1198]

King's College London, 30 Aldwych, London, WC2B 4BG, United Kingdom
{maayan.armony, albert.merono, gerard.canal}@kcl.ac.uk

Abstract. AI planning involves finding a sequence of actions that achieve a predefined goal when executed from a given initial state of the environment. Task planning is used to execute tasks specified in formal planning description languages. Ontologies to support the planning process have mainly been used in robotics; however, existing ontologies rarely provide direct mappings between planning constructs and semantic representations, and they typically do not decompose planning terms into their constituent elements. This limits expressive querying, task analysis across environments, and the integration of external knowledge. In this paper, we present TP-ONT, a novel ontology for task planning designed to address these shortcomings. TP-ONT decomposes states and predicates into their atomic components and incorporates object type hierarchies, enabling richer reasoning over relations between specified environments, actions, states, and plans. Our framework for knowledge graph and ontology construction allows automatic extension from new planning descriptions, as well as new plan solutions. Evaluation using 18 competency questions shows that the baseline ontologies can answer at most 7 questions partially or fully, whereas TP-ONT is able to answer all questions.

Keywords: Knowledge Engineering · Planning Ontology · Robotics Ontology · Ontology Alignment.

1 Introduction

AI planning research involves the search for a sequence of actions that brings a system from an initial state of the environment to a desired goal state [20]. Planning is often used in domains like robotics to enable a robot to perform a specific motion or task, or to follow a path. In task planning, task information is represented using formal description languages like PDDL [37]. The knowledge on which decision-making relies is thus what is explicitly represented in these description files. In some cases, this information may not be an expressive-enough representation of the environment. However, adding extensive further knowledge to the description can cause the search space to grow exponentially and become intractable. Moreover, the semantic meaning behind the terms used for representation are not considered in the decision-making of planners.

This gap in knowledge can be bridged by integrating external knowledge sources into the planning process. For effective knowledge integration into planning, some approaches use LLMs as a knowledge base, taking the current context into account [54, 30]. However, the information provided by LLMs cannot be validated ahead of time, and due to their black-box nature, the reasoning behind the LLM’s decision-making is harder to trace. Efforts have also long been made to design ontologies and knowledge graphs for robotics. The use of structured knowledge aids in disambiguating terms [43], and can provide verifiable and explainable information when the resources are built responsibly [53, 16]. The knowledge in the ontologies can in turn be used to reason about the world and the environment, in parallel to the planning process or the task execution, without having to directly augment the PDDL files. Some approaches use domain-specific knowledge bases to adapt the robot’s behaviour [22, 49], which improves task performance, but is difficult to maintain across domains.

Ontologies can be used at different stages of planning, to automate the creation of description files [1], as part of the planning process [42], or to analyse the plan solutions [40]. Nonetheless, existing ontologies focus on integrating knowledge during execution, rather than during the planning process itself, and the semantic representations in these ontologies are not mapped to all the planning terms that are required to formally describe tasks, making it impossible to reason about and analyse all planning components using one ontology.

Muppasani et al. [39] introduce an ontology for planning, with a focus on choosing the best planner for a task and providing explanations for plans. However, their approach does not decompose all planning terms into their atomic components, and does not explicitly represent hierarchy of object types, which limits the complexity of reasoning over domain and action reuse, and the ability to create a robust and uniform representation of environments across domains.

In this paper, we propose TP-ONT, a task planning ontology which extends upon the Planning Ontology [39]. TP-ONT decomposes plans, states, and predicates into their atomic components, to enable more fine-grained querying over planning terms, supporting more complex reasoning about constraints, and inferring new knowledge across domains. The ontology also incorporates object type hierarchies, which allows for more sophisticated comparisons between different actions, predicates, or states, and can be related to real-world and common-sense knowledge for better alignment with real environments. Our approach: (1) facilitates creating connections and drawing conclusions across planning domains, leading to generalisation and reuse of formal planning description (2) provides improved reasoning and analysis of actions and plans to find those that align better with human decision-making and (3) improves ontology alignment with popular general-knowledge and robotics ontologies compared to existing planning ontologies, which could further support complex reasoning over planning terms and help improve the overall quality of plans. We show that in our evaluation using 18 competency questions, at most 7 can be partially or fully answered with the baseline ontologies, whereas TP-ONT is able to answer all competency questions. We make all resources involved in the creation of TP-

ONT available at <https://github.com/maayan25/tp-ont> and the ontology at <https://w3id.org/tp-ont/>.

2 Related Work

2.1 General Knowledge Ontologies

General knowledge ontologies provide a structured representation of information about the world, using real-world semantics. Such ontologies include ConceptNet [47], DBPedia [5], and Schema.org [23]. Some general knowledge ontologies are designed to explicitly represent common-sense knowledge, which is knowledge that informs humans about the world. For example, Cyc [35] was a large-scale project to aggregate this type of knowledge into a knowledge base. DOLCE [19] is a top-level ontology, which provides classification into categories and general concepts, and aims to represent common-sense knowledge that humans use for reasoning on a daily basis. Common-sense knowledge can become useful for decision-making systems, including planners, by providing additional context about environment. This may be through, for example, further consideration of object characteristics based on semantics, or by utilising causal and temporal information about common events for action ordering.

2.2 Robotics Ontologies

Ontologies in Robotics. Ontologies have long been used in robotics, for representing the physical world or relations between actions, and for tracking task execution. Some approaches use domain-specific knowledge bases to change the robot’s behaviour, for example by using medical knowledge bases [22] or knowledge for search-and-rescue operations [49]. These approaches improve task performance; however, they are hard to generalise to other domains.

Other methods focus on the execution of tasks. SOMA [10] represents social and physical information about daily activities, focusing on objects’ force and motion attributes. KnowRob [50], ORKA [2], and PlanOwl [1] bridge knowledge acquired from perception of the environment with its current representation. KnowRob is based on the Cyc ontology [35], and with time was adapted to import terms from DUL [27] and SOMA [10]. The ontology additionally represents action hierarchy. KnowRob 2.0 [8] is its extension for motion planning. OCRA [40] is an ontology developed for reasoning about agent collaboration and plan-adaptation, for effective human-robot collaboration. The authors further extend it with an ontology [41] for modelling and reasoning about comparable plans, however, the ontology does not represent all planning terms. Another approach uses OWL-DL to store the state of the environment along with a common-sense ontology with related terms [34]. However, the ontology does not map to PDDL representation, and the framework relies on online interaction.

These ontologies focus on the execution of tasks rather than the planning process, and do not directly map terms from formal planning descriptions to ontology terminology. They thus cannot be leveraged for dynamically representing the available environment knowledge during planning and reasoning over it.

Planning Ontologies. Some efforts have been made to design ontologies for planning, for instance, by providing the agent with knowledge learnt from previous task executions [42], or by integrating ontologies directly into the planning process [9, 31]. Others use ontologies and DL reasoning for creating planning domains and problems [11, 32], or for representing hierarchical planning descriptions [24]. The latter explicitly defines domains and problems, as well as a type hierarchy between objects. However, they do not decompose actions, predicates, or states into their components. The DOLCE+DnS Ultralite (DUL) ontology [27] combines the Dolce ontology with other ontologies that represent situations and plans. It provides many concepts that can be mapped to concepts from the planning process, however, the relations between those do not always align with their use in planning, which could result in inconsistency. The Planning Ontology [39] is based on PDDL description [37]. The ontology is designed for choosing the best planner and for generating explanations for plans. It allows a direct translation from PDDL, however, it does not decompose all planning parts into their components, and does not represent type hierarchy.

Various approaches have shown that integration of external knowledge into planning can improve task performance [15, 13] and efficiency [54], at various points of the planning process, and for a diverse set of tasks [4].

In our approach, we address these shortcomings whilst considering the semantics of terms, towards better alignment to general ontologies. Our ontology, TP-ONT, represents the atomic components of planning description terms, allowing for more sophisticated querying and richer reasoning across domains. The ontology also represents object type hierarchy, using class hierarchy, for improved alignment with general-knowledge ontologies and for reasoning over object characteristics for better decision-making.

2.3 Ontology alignment

Ontology alignment (OA) is the task of finding mappings of corresponding concepts between two heterogeneous ontologies. Aligning ontologies aids in sharing knowledge between them and reusing knowledge for new purposes, such as integration of external knowledge into AI planning. Over the years, many methods have been suggested for matching ontologies, ranging from structural analysis of graph structure to semantic analysis of terms [3]. Recent methods include the use of large language models (LLMs) [26], retrieval-augmented generation (RAG) [6], knowledge-graph embeddings (KGE) [51], and property matching with sentence embeddings (PropMatch) [46]. The Ontology Alignment Evaluation Initiative (OAEI) [17] are annual campaigns to evaluate the most recent OA methods. OntoAligner [7] is a framework which aggregates OA methods to facilitate the use of various methods for the same task.

3 Preliminaries

3.1 Classical Planning

AI planning involves searching for a plan that achieves a given goal in a specified environment. The environment consists of a set of predicates that describe the environment’s state and a set of available actions. A problem specification specifies the agents and objects involved in it, the environment’s initial state, and the goal state. There exist different planning paradigms, such as classical planning, which assumes determinism and full observability of the environment, and more complex ones, including temporal planning and planning under uncertainty.

Actions. An action takes a set of parameters, and is defined by a name, preconditions, and effects. A grounded action is an action whose parameters are populated with objects from the problem definition. An action is applicable if its preconditions are met. In domains that specify types, a parameter is assigned an object type, which denotes the set of objects allowed to be used in it.

Predicates and propositions. Action preconditions and effects are defined using predicates. A predicate is composed of a fluent and arguments [20]. A proposition is a grounded predicate, where arguments are populated with objects from the problem definition. A state is a set of propositions.

Plans and states. A plan is a sequence of grounded actions that achieves the goal from the initial state. A plan can have a cost, where each action has a cost, and the plan’s cost being the sum of costs for all actions in the plan. Considering the execution of a plan as a transition system, each state is composed of the propositions that hold true in that state of the plan. Transitions between states occur by applying actions, whose effects modify the truth values of the propositions by adding or removing propositions. This creates a new state.

3.2 PDDL

PDDL [37] is a formal description language that encodes the information required by the planner to solve a task. It has two types of files – domain and problem files. A domain file specifies the domain name, object types in the environment (including agents), predicates to describe the state of the environment, the available actions, and their preconditions and effects. In typed domains, the types can be defined in a hierarchical structure which includes subtypes. A problem file specifies the problem name, the available objects (and their types), and the initial and goal states. PDDL domain and problem instances have often been designed and engineered as part of benchmarks for the International Planning Competitions (IPC) [28]. Many of these domains have become popular environments for the evaluation of classical planning approaches. PDDL incorporates description formalisms such as STRIPS and ADL, with many IPC domains being represented in both formalisms.

4 Ontology Design

We design and develop TP-ONT, an ontology for AI task planning. Our ontology is intended to improve reasoning and complement the task planning process. Many ontology-engineering methodologies have been developed over the years, such as Methontology [18], NeOn [48], NeOn [48], and LOT [44]. We follow the NeOn methodology for ontology engineering, as it provides design and engineering scenarios for reuse of both ontological and non-ontological resources, which are the most suitable for our use-cases. We therefore follow scenarios 2 and 4 as guidelines for building our ontology, as detailed below.

Scenario 2: Reusing and Re-engineering Non-Ontological Resources

Activities for reuse process

1. **Search non-ontological resources:** Our dataset is formed from publicly available resources of classical planning tasks, which provide PDDL files for domain and problem instances.
2. **Assess the set of candidate non-ontological resources:** We aim to collect a diverse set of domains, representing multiple types of environments. We focus on popular domains that are used in literature, such as the IPC domains [28].
3. **Select the most appropriate non-ontological resources:** We choose domains that are widely used by the planning community, based on their structure and semantics. We select semantically-rich domains that have an object type hierarchy, and represent real-world problems (rather than games, for example), and focus on STRIPS domains, for uniform parsing.

Activities for re-engineering

1. **Non-ontological resource reverse engineering:** We review and classify PDDL terms into classes and their instances, and decide what parts should be represented in the ontology.
2. **Non-ontological resource transformation:** We map PDDL terms to ontological terms, and create a pipeline to transform PDDL into an ontology.
3. **Ontology forward engineering:** We implement our ontology in OWL.

Scenario 4: Reusing and Re-engineering Ontological Resources

Choosing the most suitable ontological resources We research ontologies in Robotics and Planning, and the Planning Ontology [39] is suitable for reuse, as it directly maps PDDL terms to ontology terms.

Re-engineering process We inspect the ontology to decide which terms could be imported into our ontology, and which cannot. These decisions are detailed in the ontology’s description in the next section.

5 TP-ONT

We create an ontology that builds upon OWL and RDFS, and reuses terms from the Planning Ontology [39]. Our ontology is built in two stages – we first create a static ontology based on the terms used in AI Planning and PDDL, and then dynamically augment the static ontology with types, based on PDDL description files, in an automated process. This allows for the ontology to be further expanded with new PDDL descriptions, and to be easily maintained as new domains are introduced. This two-step process also aligns with the two scenarios we follow for designing the ontology – the static ontology extends existing ontologies, and the dynamic ontology reuses non-ontological resources to build upon the static ontology. The design choices in our ontology are motivated by the main use cases of better integration of planning ontologies to general ontologies, and of facilitating better analysis of planning components for reuse and generalisation across domains, towards open-vocabulary planning and better handling of dynamic and uncertain environments, which is naturally derived from the gap in available information between PDDL representation and actual execution of plans in the real-world environment [12, 22, 49].

5.1 Static Ontology

We reuse common planning terms from the Planning Ontology, such as the `InitialState` and `GoalState` classes, and common properties for relations between components, namely, `hasAction` and `hasParameter`.

Reuse and terminology. Where appropriate, we decouple class names from their association. For example, we define the classes `Action` and `Effect`, rather than `DomainAction` and `ActionEffect` as in the Planning Ontology, as these names should better align with general knowledge ontologies, and meaningfully convey the semantic meaning of the class. Where applicable, we create equivalence relations between our classes and classes from other planning ontologies, to support alignment. Otherwise, we introduce additional relations and classes to make further connections that can be derived from the PDDL description, as explained below. If the association in the class name is relevant, we maintain it. For instance, `PlanningDomain` and `PlanningProblem` in the Planning Ontology would not always be semantically identical to `Domain` and `Problem` classes, especially in general knowledge ontologies. However, in this case maintaining the relation to planning in the class name is appropriate. Plans in the Planning Ontology must have a `PlanCost`. As not all planners produce a cost, we do not equate the Plan classes, and `PlanCost` remains optional in our ontology.

Nominal and verbal concepts. In planning description languages, some terms are implicitly assumed as a certain part of speech. For example, actions are verbs, and objects are nouns [32]. To provide further semantic context to terms that may be ambiguous, we introduce the superclasses `VerbalConcept` and `NominalConcept`. We define `Action` as a subclass of `VerbalConcept`, and

Parameter and **Object** as subclasses of **NominalConcept**. For example, the word **Place** could be an action name, and thus a verb, but also an object type, and thus a noun.

Decomposition of predicates. In addition to recording them as entities, we decompose predicates and propositions into their fluents and arguments. For this purpose, we introduce new classes: **Fluent** and **Argument**, and new properties: **hasFluent** and **hasArgument**.

Fluents. By separating fluents from their arguments, which are often denoted using domain-specific terms, we can find commonality in fluents across domains, that are identical or found as semantically related with further analysis. This can be used to analyse similarity between domains or to real-world environments – if two domains have many predicates in common, they are more likely to be comparable, or related to each other. Similarly, we can use fluents to identify commonality between actions, by comparing actions’ preconditions and effects.

In both cases, had we not separated fluents from arguments, we would have been less likely to infer equality. Consider the PDDL syntax for a predicate, (**fluent** ?obj1 - type1 ?obj2 - type2). We can recognise similarity between predicates that are almost identical in their meaning, but the full predicate requires further analysis, such as (**at** ?obj - locatable ?loc - location) and (**in** ?obj1 - obj ?obj - truck) in *Driverlog* and (**at** ?obj - physobj ?loc - place) and (**in** ?pkg - package ?veh - vehicle) in *Logistics*. By decomposing them, we can find that the predicates share fluents, and that the argument types belong to the same type hierarchy (e.g. **package** is a subtype of **physobj**, and **truck** is a subtype of **vehicle**).

Propositions. We decompose states into their propositions, which are grounded predicates, and are instances of a new class **Proposition**. We use a property **partOf** to relate propositions to the states in which they are known to be true. The ordering of propositions in a state does not matter, so they do not need to relate to each other. We also introduce the property **hasProposition** to relate a predicate and its grounded version. For example, predicate (**in** ?pkg ?veh) in *Logistics* will have a **hasProposition** relation to (**in** p1 v1).

Plan steps. We introduce the notion of grounded actions for actions that are instantiated in a plan as a **Step** of the plan, with relations **happensBefore** and **happensAfter** to connect steps, and **hasStep** between a plan and its steps. We add a **State** class and define a relation **hasState** between a plan and its states, and **precedes** between consecutive states in a given plan, for better traceability.

Negation. In PDDL, an action may have a delete effect, written formally as **not**(**fluent** arg0 ... argN), which will remove the grounded proposition of predicate (**fluent** arg0 ... argN) from the new state of the environment. As such, rather than removing the proposition, we allow the instantiation of negative fluents (e.g. **not_holding**), which are related to their positive fluent with a **isNegationOf** relation and belong to a new **NegatedFluent** class.

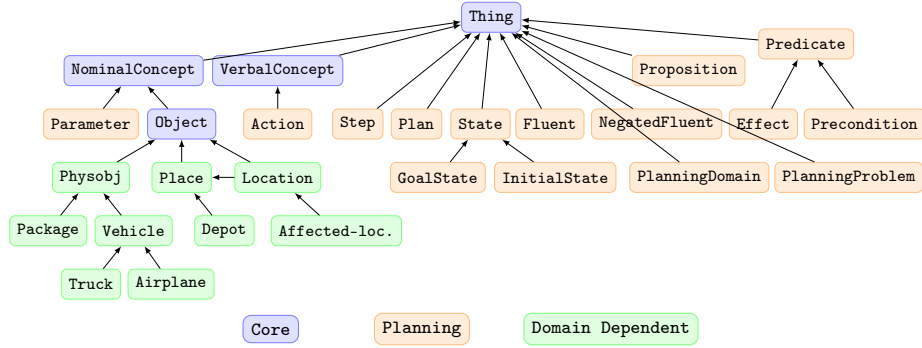


Fig. 1: Example class hierarchy; arrows denote `rdfs:subClassOf`.

5.2 Dynamic Ontology

Object types. Most PDDL domains have a typed variant, where the domain description includes object types, and often a hierarchy between them. For instance, in the Logistics domain, `Truck` and `Airplane` are subtypes of `Vehicle`, which is a subtype of `Physobj`, subtype of `Object`. These types would often represent classes of general-knowledge ontologies. Therefore, although types are domain-specific knowledge, we define them as classes in our ontology, as can be seen in Figure 1. This design contributes to cultivating further relations across domains, and to real-world objects, which contributes to aligning terms and instances with other ontologies, and allowing future integration of knowledge that could be leveraged to modify and refine the type hierarchy based on real-world knowledge.

Predicates. As discussed, we decompose predicates into their fluents and arguments. Creating relations between predicates and their fluents and arguments allows us to constrain their instantiation, for example, with the number of arguments they are allowed to take. Each predicate has `hasArg_i` properties, where `i` denotes the argument’s index. The properties are unique for each fluent in each PDDL domain, their domain is `Predicate` and the range is the argument’s object type. As such, the range of the property is defined dynamically. For example, a predicate (`in ?pkg - package ?veh - vehicle`) in Logistics will have a property `hasArg0` with domain `Predicate` and range `Package`, and a property `hasArg1` with domain `Predicate` and range `Vehicle`.

All other domain- and problem-dependent information is added to the knowledge graph as instances of the classes defined above.

6 Experimental Setup

6.1 Ontology and KG Construction

We parse the PDDL files using the Unified-Planning framework [38], and construct a knowledge graph (KG) in an automated procedure using the RDFLib library [33] in a Python module. We execute the KG constructor, which reads the static ontology from an OWL file and, either produces a new KG for a particular domain and problem, or, if one already exists, augments it. The KG constructor is executed on seven representative domains which were chosen during the ontology design process. The dynamic part of the ontology is influenced by the domains chosen, as object types are automatically added as classes to the KG. Adding new domains and problems to the KG can be done by rerunning the script with the new PDDL files as input. For generating plan instances, we use fast-downward [25], a popular planner for classical planning, to produce optimal plans using the A* search algorithm. To simulate plan states, we utilise VAL [29], which provides the grounded effects for each action, and apply the effects using Python functions. We run the constructor for one problem per domain, as our focus is on the ontology, which is only dependent on the domain files. From the resulting KG, we extract the conceptual model and save it as a separate OWL file, which includes the dynamic ontology.

6.2 Ontology Alignment

We utilise the OntoAligner framework [7], a Python library for aligning ontologies. Where possible, we use the built-in pipeline provided in OntoAligner, which runs a parser, an encoder, and the aligner of choice. For other approaches, we design a module which runs all these components at once, based on a pre-defined configuration. We use only the Planning Ontology¹ as a baseline, and not DUL, as DUL is built for a broader context based on general ontologies, and cannot directly be used for planning without modification. We perform alignment of TP-ONT and the baseline with the following general-knowledge and robotics ontologies: DOLCE², Schema.org³, Knowrob⁴, OCRA⁵, and SOMA⁶. We use four state-of-the-art methods for alignment which are supported by OntoAligner and are diverse in their approach: KG-embeddings, PropMatch, LLM-based, and RAG-based. We use Qwen-3 8B⁷ as an additional LLM-based approach alignment, which is an LLM comparable in size to the default Mistral 7B⁸ model.

¹ <https://purl.org/ai4s/ontology/planning>

² <https://w3id.org/DOLCE/OWL/DOLCEbasic>

³ <https://schema.org/docs/developers.html>

⁴ <http://knowrob.org/kb/knowrob.owl>

⁵ <https://www.iri.upc.edu/groups/perception/OCRA/ont/ocra.owl>

⁶ <http://www.ease-crc.org/ont/SOMA.owl>

⁷ <https://huggingface.co/Qwen/Qwen3-8B>

⁸ <https://huggingface.co/mistralai/Mistral-7B-v0.3>

Armony, M. et al. (2026). <i>TP-ONT Ontology</i> . https://w3id.org/tp-ont/	
Resource	Link
Resource namespace	https://w3id.org/tp-ont/
Github organisation & code	https://github.com/maayan25/tp-ont
HuggingFace dataset	https://huggingface.co/datasets/ma25/tp-ont
Zenodo repository	https://doi.org/10.5281/zenodo.21495382
Sample SPARQL queries	https://github.com/maayan25/tp-ont/tree/main/sparql/examples

Table 1: Links to key resources of the TP-ONT ontology.

7 Evaluation

We validate the ontology using the ontology validation tool OOPS! [45]. OOPS! detects some `owl:equivalentClass` alignments to the Planning Ontology as potentially incorrect. All class pairs share the same constraints, and were verified as consistent with the HermiT reasoner. We run the HermiT reasoner [21] over our ontology and KG, which it finds consistent. We compose SPARQL queries representing our list of competency questions, and run those using Apache Jena Fuseki⁹ server, OWL-RL¹⁰ and SPARQLWrapper¹¹ in Python. The SPARQL queries can be found with the Sample SPARQL Queries link, as in Table 1.

7.1 Competency Questions

We define the following competency questions, which our ontology should be able to answer. The competency questions were chosen according to similar criteria as in other established robotics ontologies, such as Planning Ontology [39] and OCRA [40], and designed to be of interest to the planning community in the context of knowledge integration. The questions were defined iteratively, whilst revising the design of the ontology and taking into consideration its purpose, such that a system that is able to answer these questions can perform a range of reasoning and analysis tasks over domain knowledge and generalisation of PDDL domains. The questions vary in their aim, exploring the usability of the various design aspects of our ontology, including: type hierarchy (Q12) across domains (Qs 1,4,8,11,13), general PDDL functionality (Q2), commonalities across domains and actions (Qs 3,6-7), predicate decomposition (Qs 5,11,14-16), and plan decomposition and analysis (Qs 9-10,16-18). Table 2 details whether, for each question, it can be answered using the Planning Ontology [39] or DUL [27], along an explanation.

1. What object types are of type O?
2. What actions exist in domain D?
3. What domains have an action A?

⁹ <https://jena.apache.org/documentation/fuseki2/index.html>

¹⁰ <https://owl-rl.readthedocs.io/en/latest/owlrl.html>

¹¹ <https://sparqlwrapper.readthedocs.io/en/latest/main.html>

4. What domains have objects of type T or its subtypes?
5. How many arguments does predicate P take in domain D?
6. What predicates have more than N arguments?
7. What are the preconditions of action A, across domains?
8. What subtype of type T is most common across domains?
9. Which action was taken right before action A in plan P?
10. In problem P, which objects are in location L at the initial state?
11. Does predicate P take the same object type for the Nth arg. in D1 and D2?
12. Does domain D have multiple object types of type T?
13. Do multiple domains have object types of type T?
14. What object type can be the first argument of predicate P in D?
15. What are the three most common fluents across domains?
16. Which objects are in location L at any point in plan P?
17. Are there common action patterns in plans for domain D?
18. Which domain has typically longer plans, D1 or D2?

CQ	Planning Ontology	DUL	CQ	Planning Ontology	DUL
1.	N Type hierarchy	N Type hierarchy	11.	N Predicate decomposition	N No predicates
2.	Y D hasAction A	Y D includesAction A	12.	N Type hierarchy	N Type hierarchy
3.	Y D hasAction A	Y D includesAction A	13.	Y They declare types	N No type hierarchy
4.	P Type hierarchy	P Type hierarchy	14.	N Predicate decomposition	N No predicates
5.	N Predicate decomposition	N No predicates	15.	N No fluents	N No fluents
6.	N Predicate decomposition	N No predicates	16.	N Proposition decomposition	N States not related to objects
7.	Y A has Precondition P	N No predicates	17.	N Proposition decomposition	P If define Process as a set of Actions
8.	P Yes, but not subtypes	N Type hierarchy	18.	P When action cost is uniform	P If plan is a Process
9.	N Plan decomposition	P Action ordering not plan-specific			
10.	N Proposition decomposition	N States not related to objects			

Table 2: Can CQs be answered by the baselines: N - No, P - partially, Y - yes

7.2 Use Case Scenarios

In this section we present use-cases which can be addressed by TP-ONT, but not with the baseline ontologies.

Towards open-vocabulary and universal Planning. Transport domains, such as Logistics, Depot, and Driverlog, often contain an action for unloading an object from a vehicle. However, the naming conventions of the action and its components can differ across domains, and types may be stricter in some than others. For example, in Logistics, the Unload action is split to two actions, one for airplanes, and one for trucks, which require an object of type **Airplane** or **Truck**, respectively, as their parameters. Using our ontology, we are able to generalise unload actions across domains, and infer that while these are separate actions, their preconditions, and effects are typically structurally similar, and the object types they require belong to the same class (vehicles and objects). We can thus create a general action with the following steps:

1. Query the ontology for all unload actions, using a regular expression.
2. Extract preconditions and effects using the `hasPrecondition` and `hasEffect` properties.
3. Find common fluents among preconditions using `hasFluent` (e.g. `at`, and check the types of their arguments, with `hasArg0` and `hasArg1` and `rdf:type`.
4. Identify the type hierarchy, and the object type for each parameter in the generalised version.

We can gather that **Truck** and **Airplane** are both subtypes of **Vehicle**, and that **Package** and **Object** are of type **Object**, making the generalised action in Table 3 directly derivable. The new **Unload** action can be used in a new domain, and the parameter types can be those in the generalised version, or any of their subtypes, which can also be inferred from the ontology. With this approach we can support tasks like action inheritance [36], which is used to abstract actions for having multiple versions of the same domain, with our ontology allowing to do so dynamically and without having to modify the PDDL files. Another application could be to support dynamic creation of domains [14], or translation of actions from natural language to PDDL [52], by providing actions based on names and parameter object types, rather than creating those from scratch.

Action	Domain	Preconditions	Effects
Unload	Depot	<i>at _x_p, at _z_p, avail _x, in _y_z</i>	<i>lifting _x_y, not _avail _x, not _in _y_z</i>
Unload-airplane	Logistics	<i>at _airplane_loc, in _pkg_airplane</i>	<i>at _pkg_loc, not _in _pkg_airplane</i>
Unload-truck	Logistics	<i>at _truck_loc, in _pkg_truck</i>	<i>at _pkg_loc, not _in _pkg_truck</i>
Unload-truck	Driverlog	<i>at _truck_loc, in _obj_truck</i>	<i>at _obj_loc, not _in _obj_truck</i>
Unload	Generalised	<i>at _vehicle_loc, in _obj_vehicle</i>	<i>at _obj_loc, not _in _obj_vehicle</i>

Table 3: Unload-type actions across domains and their generalisation.

Knowledge Integration

Knowledge integration for better type hierarchy. Owing to the design of our ontology, we can use information from external knowledge sources to correct or extend the existing typing hierarchies, and unify them across PDDL domains. Grounding the PDDL type hierarchy in general and common-sense knowledge can help us in extending and merging domains, in applying additional constraints on action orders, and in adding preconditions or effects based on semantics, context, and common knowledge.

For example, we can use external knowledge on type hierarchy to improve consistency in hierarchy across domains. In our ontology, **Location** is a subclass of **Place** and of **Object**, as it is specified differently in individual domain specifications (as illustrated in Figure 1). A more robust and expressive type hierarchy can support advancing towards generalisable environment specifications, and enable sharing of constraints across similar environments. For example, we

can derive from common-sense knowledge that all objects of type **Vehicle**, in any realistic domain, are always in a location. This means that every action involving vehicles must have a precondition relating the vehicle to a parameter of type **Location**. More formally, if a **Parameter** of an **Action** is of type **Vehicle**, it must have a **Precondition** that has an **Argument** of type **Vehicle** and an **Argument** of type **Location**. Having a more rigid and uniform typing system across domains can support reusability and thus, in addition to more effective domain and construction, constraints across domains can also be shared and adapted, reducing the search space.

We can also determine constraints from common-sense knowledge that domain designers usually consider implicitly, or must explicitly define in the domain description. In PDDL, an object can typically only be an instance of one type, and a type has only one direct parent. Using TP-ONT’s representation of types, we can maintain this structure for the original object type and use it to enforce strict constraints introduced in the description files (such as which object types are allowed for an argument in a predicate), and additionally infer common characteristics across types, and use those as further soft constraints, to navigate search or choose between possible plans. For example, the **Locatable** class has various subtypes across domains, with some of these being things that move autonomously (e.g. **Driver**) and some that don’t (e.g. **Surface**). We can introduce new parent classes such as **AutonomousAgent** and **Stationary** dynamically, based on acquired external knowledge on the objects involved in a problem, and use those to infer new assertions about those classes. For example, we may be able to infer that the location of an object belonging to the **AutonomousAgent** class may move independently of the action we take, and its location may not be known to us (which diverges from classical planning to planning with uncertainty), and for a **Stationary** we can expect its location to stay the same unless it is moved by another agent.

Knowledge integration for improved decision-making. Utilising the type hierarchy, we can use common-sense knowledge to modify or augment the ontology, aiming to align the decision-making process with human decision-making. Planning problems often have multiple optimal plans, that is, plans with the same lowest-possible cost, and based on the domain description they are of the same quality. However, some actions in those plans may be more appropriate than others depending on the semantic meaning behind actions and objects, the state of the environment or the previous actions taken. For instance, in a healthcare domain that has an action to transport patients to receive medical attention, such as [12], there could be multiple safe areas (subtypes of **Location**) for an ambulance (subtype of **Vehicle**) to stop to treat the patients. The ideal location to transport the patient to will often depend on the context, for instance how urgent the care is. **Hospital** is likely to always be an appropriate choice, but may not always be the best one, for example if it’s too far away, or the injury is minor. Providing a strict constraint on having the location be a type of **Hospital** would not always provide the best plans. Other location types like **Home** and **SideOfRoad** could be appropriate for certain cases, but not for oth-

Ontology	Target	Best Method	Matchings	Coverage (%)	Agreement (count)	Agreement (%)
PlanOnto	DOLCE	LLM Mistral-7B	108	5.68	9	0.08
TP-ONT	DOLCE	LLM Mistral-7B	517	8.76	56	0.11
PlanOnto	KnowRob	LLM Mistral-7B	416	21.89	17	0.04
TP-ONT	KnowRob	LLM Mistral-7B	2324	39.39	94	0.04
PlanOnto	OCRA	LLM Mistral-7B	25	1.32	7	0.28
TP-ONT	OCRA	LLM Mistral-7B	86	1.46	14	0.16
PlanOnto	SOMA	LLM Mistral-7B	1094	57.58	24	0.02
TP-ONT	SOMA	LLM Mistral-7B	4780	81.02	155	0.03
PlanOnto	Schema.org	LLM Mistral-7B	2127	111.95	28	0.01
TP-ONT	Schema.org	LLM Mistral-7B	9765	165.51	201	0.02

Table 4: OA comparison between TP-ONT and the Planning Ontology

ers. Thus, no subtype could suffice as a constraint in this case, but providing **Location** as the type is too broad. Using external knowledge from knowledge sources, we are able to access information on demand, without encoding this knowledge in planning description, which may be a more appropriate approach to knowledge integration for complex tasks [4]. This could be done, for instance, by creating dynamic typing constraints on some actions and predicates, that we are able to analyse thanks to the representation of parameters and arguments.

7.3 Evaluation of Ontology Alignment

Table 4 details preliminary results on the ontology alignment (OA) of planning ontologies with general-knowledge and robotics ontologies, for the best alignment method, which is LLM-Based alignment using Mistral-7B. We can see that the total matchings and coverage are consistently higher for TP-ONT than that of the Planning Ontology. We compute the coverage as the total number of classes from the source ontology that appear in at least one matching, divided by the total number of classes. The agreement is calculated as the number of alignment pairs (source onto x target onto) that have been found by more than one alignment method from all alignment pairs. The OA evaluation is provided as an additional assessment of the resource, as a first step towards further research on the impact of integrating external knowledge in planning. Considering the use of OntoAligner and most-recent alignment methods, the findings provide a reliable first baseline.

8 Discussion

Our results show that using TP-ONT we are able to answer multiple competency questions that cannot be answered by the baseline ontologies, thanks to the type hierarchy and term decomposition. The results of the SPARQL queries aligned with these questions can help analyse commonalities across actions and domains, and to generalise knowledge about represented environments. They can

also help to more effectively generate new domains which are built for a specific environment, but have common attributes with multiple other environments, for example, a new Transport domain which uses a certain vehicle and can carry a specific type of objects. The use-case scenarios we have introduced demonstrate the applicability and reusability of our approach for various purposes and its potential contributions to the semantic-web and planning communities. The stronger alignment of our ontology compared to the baseline shows that TP-ONT’s design leverages the structure of PDDL representation, allowing a better integration with heterogeneous ontologies across both general knowledge and robotics-specific knowledge. This integration enables complex semantic reasoning over planning components, grounded in real-world knowledge about the environment, to better align plans with human expectations.

Finally, our framework facilitates automatic and dynamic creation of the KG and extraction of the ontology, increasing its reproducibility, and supporting maintainability of the resource by the community, allowing simple adaptation to specific needs and promoting the integration of ontologies into planning.

9 Conclusions

In this work, we have presented TP-ONT, a task planning ontology which decomposes plans, states, and predicates into their atomic components, and incorporates object type hierarchies. This design enables more fine-grained querying over planning terms, supporting more complex reasoning about constraints, and inferring new knowledge across domains as well as more sophisticated comparison between different actions, predicates, or states. Our evaluation demonstrates that the design of the ontology facilitates better alignment with general-knowledge ontologies, which could further support complex reasoning over planning terms and help improve the overall quality of plans.

The ontology is planned to be maintained, extended, and used in future projects. Further work on the ontology could include extending the parsing and KG construction to support planning paradigms beyond classical planning, such as temporal planning, where actions have durations and temporal constraints, and planning under uncertainty, where the environment may not be deterministic.

Acknowledgments. This work has been supported by UK Research and Innovation (EP/S023356/1), in the UKRI Centre for Doctoral Training in Safe and Trusted Artificial Intelligence (www.safeandtrustedai.org). This work has also been supported by King’s College London (2025), King’s Computational Research, Engineering and Technology Environment (CREATE). Retrieved March 22, 2025, from <https://doi.org/10.18742/rnvf-m076>. We thank the anonymous reviewers for their feedback.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Declaration of use of Generative AI Claude was used to generate skeleton of figures and of functions to create tables.

References

1. Adamik, M., Forte, P.: Planowl: Automated pddl files generation from owl ontologies and visual language models. In: *Proceedings of the AAAI Symposium Series*. vol. 7, pp. 634–643 (2025)
2. Adamik, M., Pernisch, R., Tiddi, I., Schlobach, S.: Orka: An ontology for robotic knowledge acquisition. In: *International Conference on Knowledge Engineering and Knowledge Management*. pp. 309–327. Springer (2024)
3. Ardjani, F., Bouchiha, D., Malki, M.: Ontology-alignment techniques: survey and analysis. *International Journal of Modern Education and Computer Science* **7**(11), 67 (2015)
4. Armony, M., Meroño-Peñuela, A., Canal, G.: When can planning benefit from common-sense knowledge? In: *ICAPS Workshop on Knowledge Engineering for Planning and Scheduling (KEPS)* (2026)
5. Auer, S., Bizer, C., Kobilarov, G., Lehmann, J., Cyganiak, R., Ives, Z.: Dbpedia: A nucleus for a web of open data. In: *international semantic web conference*. pp. 722–735. Springer (2007)
6. Babaei Giglou, H., D’Souza, J., Engel, F., Auer, S.: Llms4om: Matching ontologies with large language models. In: *European Semantic Web Conference*. pp. 25–35. Springer (2024)
7. Babaei Giglou, H., D’Souza, J., Karras, O., Auer, S.: Ontoaligner: A comprehensive modular and robust python toolkit for ontology alignment. In: *European Semantic Web Conference*. pp. 174–191. Springer (2025)
8. Beetz, M., Bekler, D., Haidu, A., Pomarlan, M., Bozcuoğlu, A.K., Bartels, G.: Know rob 2.0—a 2nd generation knowledge processing framework for cognition-enabled robotic agents. In: *2018 IEEE international conference on robotics and automation (ICRA)*. pp. 512–519. IEEE (2018)
9. Behnke, G., Bercher, P., Biundo, S., Glimm, B., Ponomaryov, D., Schiller, M.: Integrating ontologies and planning for cognitive systems. In: *Proceedings of the 28th International Workshop on Description Logics (DL 2015), CEUR Workshop Proceedings*. pp. 338–360 (2015)
10. Bekler, D., Porzel, R., Pomarlan, M., Vyas, A., Höffner, S., Beetz, M., Malaka, R., Bateman, J.: Foundations of the socio-physical model of activities (soma) for autonomous robotic agents 1. In: *Formal ontology in information systems*. pp. 159–174. IOS Press (2021)
11. Bouillet, E., Feblowitz, M., Liu, Z., Ranganathan, A., Riabov, A.: A knowledge engineering and planning framework based on owl ontologies. *Proceedings of the Second International Competition on Knowledge Engineering* **191** (2007)
12. Chrpá, L., Vallati, M.: On the exploitation of automated planning for efficient decision making in road traffic accident management. In: *2016 IEEE 55th Conference on Decision and Control (CDC)*. pp. 6607–6612. IEEE (2016)
13. Conti, C.J., Varde, A.S., Wang, W.: Robot action planning by commonsense knowledge in human-robot collaborative tasks. In: *2020 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS)*. pp. 1–7. IEEE (2020)
14. Cresswell, S.N., McCluskey, T.L., West, M.M.: Acquiring planning domain models using locm. *The Knowledge Engineering Review* **28**(2), 195–213 (2013)

15. Ding, Y., Zhang, X., Amiri, S., Cao, N., Yang, H., Kaminski, A., Esselink, C., Zhang, S.: Integrating action knowledge and llms for task planning and situation handling in open worlds. *Autonomous Robots* **47**(8), 981–997 (2023)
16. Draschner, C.F., Jabeen, H., Lehmann, J.: Ethical and sustainability considerations for knowledge graph based machine learning. In: AIKE. pp. 53–60 (2022)
17. Euzenat, J., Meilicke, C., Stuckenschmidt, H., Shvaiko, P., Trojahn, C.: Ontology alignment evaluation initiative: six years of experience. In: *Journal on data semantics XV*, pp. 158–192. Springer (2011)
18. Fernández-López, M., Gómez-Pérez, A., Juristo, N.: Methontology: From ontological art towards ontological engineering. In: *Proceedings of the AAAI Spring Symposium on Ontological Engineering*. pp. 33–40 (1997)
19. Gangemi, A., Guarino, N., Masolo, C., Oltramari, A., Schneider, L.: Sweetening ontologies with dolce. In: *International conference on knowledge engineering and knowledge management*. pp. 166–181. Springer (2002)
20. Ghallab, M., Nau, D., Traverso, P.: *Automated Planning: theory and practice*. Elsevier (2004)
21. Glimm, B., Horrocks, I., Motik, B., Stoilos, G., Wang, Z.: Hermit: an owl 2 reasoner. *Journal of automated reasoning* **53**(3), 245–269 (2014)
22. Gonçalves, P.J., Torres, P.M.: Knowledge representation applied to robotic orthopedic surgery. *Robotics and Computer-Integrated Manufacturing* **33**, 90–99 (2015)
23. Guha, R.V., Brickley, D., Macbeth, S.: Schema. org: evolution of structured data on the web. *Communications of the ACM* **59**(2), 44–51 (2016)
24. Hartanto, R., Hertzberg, J.: Fusing dl reasoning with htn planning. In: *Annual Conference on Artificial Intelligence*. pp. 62–69. Springer (2008)
25. Helmert, M.: The fast downward planning system. *Journal of Artificial Intelligence Research* **26**, 191–246 (2006)
26. Hertling, S., Paulheim, H.: Olala: Ontology matching with large language models. In: *Proceedings of the 12th knowledge capture conference 2023*. pp. 131–139 (2023)
27. Hitzler, P., et al.: Dolce+ d&s ultralite and its main ontology design patterns. *Ontog Eng Ontog Des Patterns Found Appl* **25**, 81 (2016)
28. Hoffmann, J., Edelkamp, S., Thiébaux, S., Englert, R., Liporace, F., Trüg, S.: Engineering benchmarks for planning: the domains used in the deterministic part of ipc-4. *Journal of Artificial Intelligence Research* **26**, 453–541 (2006)
29. Howey, R., Long, D., Fox, M.: Val: Automatic plan validation, continuous effects and mixed initiative planning using pddl. In: *16th IEEE International Conference on Tools with Artificial Intelligence*. pp. 294–301. IEEE (2004)
30. Izquierdo-Badiola, S., Canal, G., Rizzo, C., Alenyà, G.: PlanCollabNL: Leveraging Large Language Models for Adaptive Plan Generation in Human-Robot Collaboration. In: *IEEE International Conference on Robotics and Automation (ICRA)*. pp. 17344–17350 (5 2024). <https://doi.org/10.1109/ICRA57147.2024.10610055>
31. John, T., Koopmann, P.: Planning with owl-dl ontologies. In: *27th European Conference on Artificial Intelligence, ECAI 2024*. pp. 4165–4172. IOS Press BV (2024)
32. Kaiser, P., Lewis, M., Petrick, R.P., Asfour, T., Steedman, M.: Extracting common sense knowledge from text for robot planning. In: *2014 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 3749–3756. IEEE (2014)
33. Krech, D., Grimnes, G.A., Higgins, G., Car, N., Hees, J., Aucamp, I., Lindström, N., Arndt, N., Sommer, A., Chuc, E., Herman, I., Nelson, A., McCusker, J., Gillespie, T., Kluyver, T., Ludwig, F., Champin, P.A., Watts, M., Holzer, U., Summers, E., Morriss, W., Winston, D., Perttula, D., Kovacevic, F., Chateaneu, R., Solbrig, H., Cogrel, B., Stuart, V.: *RDFLib* (Oct 2025). <https://doi.org/10.5281/zenodo.6845245>, <https://github.com/RDFLib/rdfLib>

34. Lemaignan, S., Ros, R., Mösenlechner, L., Alami, R., Beetz, M.: Oro, a knowledge management platform for cognitive architectures in robotics. In: 2010 IEEE/RSJ International conference on intelligent robots and systems. pp. 3548–3553. IEEE (2010)
35. Lenat, D.B.: Cyc: A large-scale investment in knowledge infrastructure. *Communications of the ACM* **38**(11), 33–38 (1995)
36. Lindsay, A.: On using action inheritance and modularity in pddl domain modelling. In: *Proceedings of the International Conference on Automated Planning and Scheduling*. vol. 33, pp. 259–267 (2023)
37. McDermott, D., Ghallab, M., Howe, A., Knoblock, C., Ram, A., Veloso, M., Weld, D., Wilkins, D.: *PDDL - The Planning Domain Definition Language* (1998)
38. Micheli, A., Bit-Monnot, A., Röger, G., Scala, E., Valentini, A., Framba, L., Rovetta, A., Trapasso, A., Bonassi, L., Gerevini, A.E., Iocchi, L., Ingrand, F., Köckemann, U., Patrizi, F., Saetti, A., Serina, I., Stock, S.: Unified planning: Modeling, manipulating and solving ai planning problems in python. *SoftwareX* **29**, 102012 (2025). <https://doi.org/https://doi.org/10.1016/j.softx.2024.102012>, <https://www.sciencedirect.com/science/article/pii/S2352711024003820>
39. Muppasani, B., Pallagani, V., Srivastava, B., Mutharaju, R.: Building and using a planning ontology from past data for performance efficiency. In: *Proc. ICAPS’23 Workshop PLanning And onTology wOrkshop (PLATO)* (2023)
40. Olivares-Alarcos, A., Foix, S., Borgo, S., Alenyà, G.: Odra—an ontology for collaborative robotics and adaptation. *Computers in Industry* **138**, 103627 (2022)
41. Olivares-Alarcos, A., Foix, S., Borràs, J., Canal, G., Alenyà, G.: Ontological foundations for contrastive explanatory narration of robot plans. *Information Sciences* **741**, 123280 (2026). <https://doi.org/10.1016/j.ins.2026.123280>
42. Pang, W., Gu, W., Li, H.: Ontology-based task planning for autonomous unmanned system: framework and principle. In: *Journal of Physics: Conference Series*. vol. 2253, p. 012018. IOP Publishing (2022)
43. Peng, C., Xia, F., Naseriparsa, M., Osborne, F.: Knowledge graphs: Opportunities and challenges. *Artificial intelligence review* **56**(11), 13071–13102 (2023)
44. Poveda-Villalón, M., Fernández-Izquierdo, A., Fernández-López, M., García-Castro, R.: Lot: An industrial oriented ontology engineering framework. *Engineering Applications of Artificial Intelligence* **111**, 104755 (2022)
45. Poveda-Villalón, M., Gómez-Pérez, A., Suárez-Figueroa, M.C.: OOPS! (Ontology Pitfall Scanner!): An On-line Tool for Ontology Evaluation. *International Journal on Semantic Web and Information Systems (IJSWIS)* **10**(2), 7–34 (2014)
46. Sousa, G., Lima, R., Trojahn, C.: Combining word and sentence embeddings with alignment extension for property matching. In: *OM@ ISWC*. pp. 91–96 (2023)
47. Speer, R., Chin, J., Havasi, C.: Conceptnet 5.5: An open multilingual graph of general knowledge. In: *Proceedings of the AAAI conference on artificial intelligence*. vol. 31 (2017)
48. Suárez-Figueroa, M.C., Gómez-Pérez, A., Fernández-López, M.: The neon methodology for ontology engineering. In: *Ontology engineering in a networked world*, pp. 9–34. Springer (2011)
49. Sun, X., Zhang, Y., Chen, J.: High-level smart decision making of a robot based on ontology in a search and rescue scenario. *Future Internet* **11**(11), 230 (2019)
50. Tenorth, M., Beetz, M.: Knowrob—knowledge processing for autonomous personal robots. In: 2009 IEEE/RSJ international conference on intelligent robots and systems. pp. 4261–4266. IEEE (2009)
51. Wang, Z.: Amd results for oaei 2022. In: *OM@ ISWC*. pp. 145–152 (2022)

52. Xie, Y., Yu, C., Zhu, T., Bai, J., Gong, Z., Soh, H.: Translating natural language to planning goals with large-language models. arXiv preprint arXiv:2302.05128 (2023)
53. Zhang, B., Koutsiana, E., Zhao, Y., Meroño-Peñuela, A., Simperl, E.: Trustworthy knowledge graphs: Practices and approaches. In: Handbook on Neurosymbolic AI and Knowledge Graphs, pp. 363–384. SAGE Publications 1 Oliver’s Yard, 55 City Road, London, EC1Y 1SP (2025)
54. Zhao, Z., Lee, W.S., Hsu, D.: Large language models as commonsense knowledge for large-scale task planning. *Advances in neural information processing systems* **36**, 31967–31987 (2023)